lsp: https://neovim.io/doc/user/lsp.html#LSP
lspconfig: https://github.com/neovim/nvim-lspconfig

==========
======
== Python ==

pylsp (Python Language Server)
https://github.com/python-lsp/python-lsp-server

Install

```
$ brew install python-language-server
```

Minimal config

```
vim.lsp.config('pylsp', {
  settings = {
    pylsp = {
      cmd = { "pylsp" }
    }
  }
})
```

* usage: using pylsp with uv or virtual environment
Problem: packages installed in **virtual environment** is not visible to LSP server by default
Solution: install pylsp in virtual environment

```
# Install LSP server in virtual
$ source .venv/bin/activate
$ uv pip install "python-lsp-server[all]"      # all=all
options
$ which pylsp
.venv/bin/pylsp                                # should
be virtual env

$ nvim main.py                                 # check
nvim
```

* usage: pylsp not working
Problem:
Solution:
Check the log

$ cat ~/.local/state/nvim/lsp.log



==========
======

== Haskell LSP with HLS (Haskell Language server) ==

Haskell language server (HLS): https://github.com/haskell/haskell-language-server

Features
Ref: https://haskell-language-server.readthedocs.io/en/latest/features.html
- completion
 ghcide: https://github.com/haskell/ghcide
- formatter
 ormolu (default):
- lint:
 hlint: https://github.com/ndmitchell/hlint

1) Install hls

```
$ brew install haskell-language-server
```

2) Configure
lspconfig: https://github.com/neovim/nvim-lspconfig/blob/master/doc/server_configurations.md#hls

```
lua <<EOF
local nvim_lsp = require('lspconfig')

vim.lsp.enable("hls")
vim.lsp.config("hls", {
  -- default
  filetypes = { "haskell", "lhaskell" }
  cmd = { "haskell-language-server-wrapper", "--lsp" },

  root_dir = function(bufnr, on_dir)
    local fname = vim.api.nvim_buf_get_name(bufnr)
    on_dir(nvim_lsp.util.root_pattern('hie.yaml', 'stack.yaml',
'cabal.project', '*.cabal', 'package.yaml')(fname))
  end,

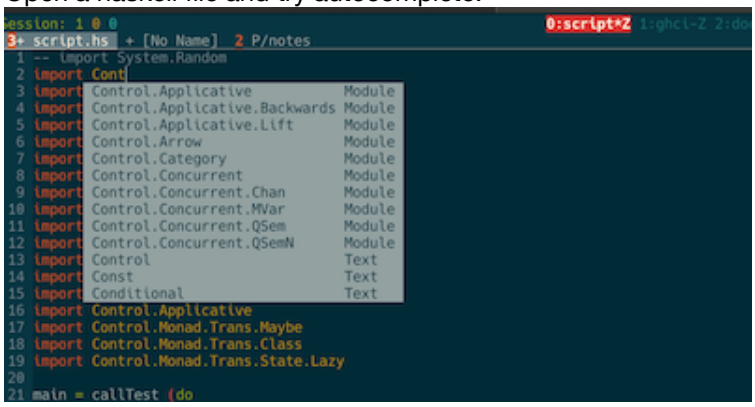
  -- [DEPRECATED: < nvim 0.11]
  root_dir = function (filepath)
   return (
     nvim_lsp.util.root_pattern('hie.yaml', 'stack.yaml',
'cabal.project')(filepath)
       or nvim_lsp.util.root_pattern('*.cabal', 'package.yaml')
(filepath)
     )
```

```
  end,

  settings = {
     haskell = {
       formattingProvider = "ormolu"
     }
  }
})

EOF
```

3) Test
Open a haskell file and try autocomplete.



[Stack] use with Stack
Ref: https://haskell-language-server.readthedocs.io/en/latest/configuration.html
Stack installs a **GHC locally** per project.
i.e. not a system-wide GHC
So HLS needs to know where to find GHC.
But HSL should already have automatic Stack support

Step 2) Setup Stack
Set **GHC version** supported by the language server

```
# ./stack.yml
resolver: ghc-9.2.4
```

Set dependencies in the **package file** (package.yml)

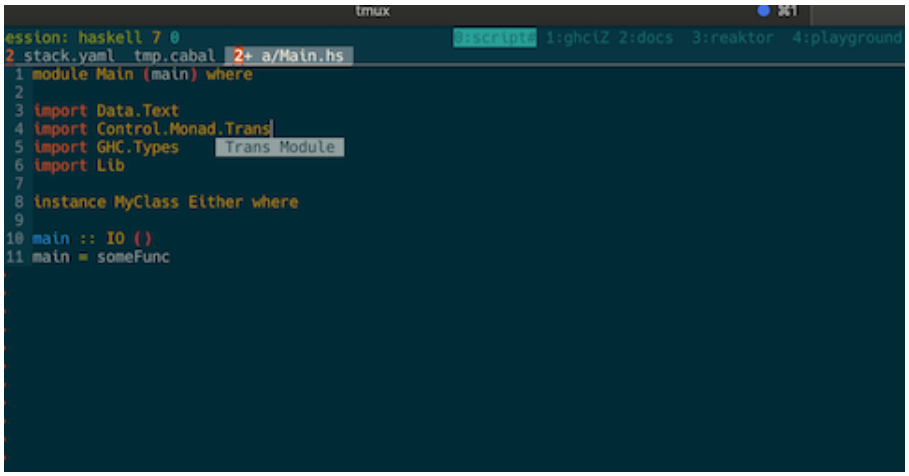
```
# ./package.yml
name:                tmp
version:             0.1.0.0
github:              "githubuser/tmp"
…

dependencies:
- base >= 4.7 && < 5
- text
- ghc-prim
- mtl
```

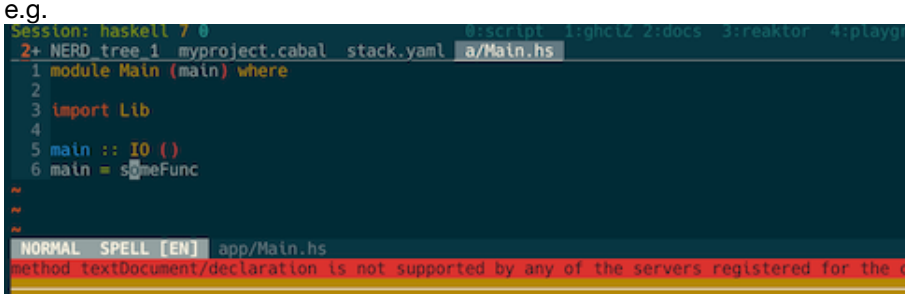
Step 2) Build stack

```
$ stack build
```

Step 3) Verify



[Stack] does not recognize files in a subdirectory e.g. error "method textDocument/declaration is not supported by any of the servers registered"
Ref: https://haskell-language-server.readthedocs.io/en/latest/configuration.html
hie-bios: https://github.com/haskell/hie-bios
e.g.



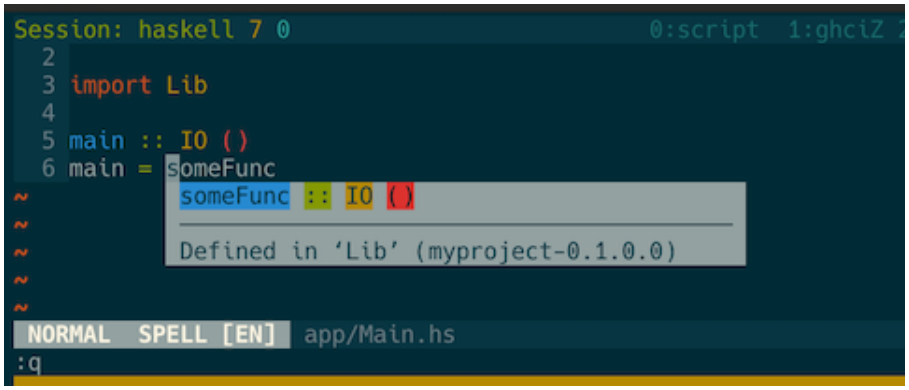
Problem: HLS does not know where to find the **source code** for **modules** aka **components**.
Solution: Need to tell HLS where to find the components via **hie-bios** using a **hie.yaml** file

Solution 1: for simple cases, just need to specify in **hie.yaml** file to use stack

Step 1) Create hie.yaml file

```
# ./hie.yaml
cradle:
 stack:
```

Step 2) Test



Solution 2: explicitly specify **component paths**
e.g. suppose project is **myproject**
Files

```
./src/Lib.hs
./app/App.hs
```

package.yml

```
name:                myproject
version:             0.1.0.0
library:
  source-dirs: src
executables:
  myMain:
    main:                Main.hs
    source-dirs:         app
```

Step 1) Create **hie.yaml**
a) Find the **components**

```
$ stack ide targets
myproject:lib
myproject:exe:myMain
```

b) Add to hie.yaml

```
# ./hie.yaml
cradle:
  stack:
    - path: "./src"
      component: "myproject:lib"
    - path: "./app"
      component: "myproject:exe:myMain"
```

Step 2) Test that the hie.yaml is **valid**
Try running HLS on some code.

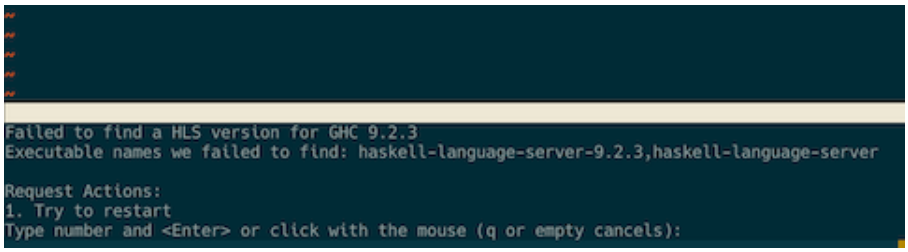
```
 $ haskell-language-server-9.2.4 --debug app/Lib.hs        #
Should succeed
 $ haskell-language-server-9.2.4 --debug app/Main.hs       #
Should succeed
```

Step 3) Verify



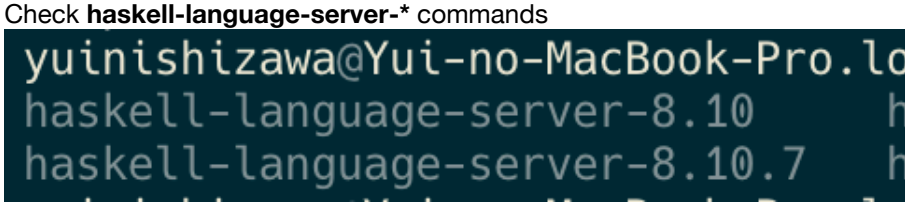
[Stack] Troubleshooting: Error "Failed to find a HLS version" when using Stack
Need to use a **GHC version** supported by HLS

e.g. Error



Step 1) Check supported **GHC versions**

Check **haskell-language-server-*** commands



Step 2) Set Stack to use a supported **GHC version**

```
# In ./stack.yml

# Solution 1) Pick environment with supported GHC version
resolver: nightly-2022-10-27        # By snapshot. Uses ghc-9.2.4.
resolver: ghc-9.2.4                 # By GHC version explicitly

# Solution 2) override compiler only
compiler: ghc-9.2.4
```

Step 3) Rebuild Stack

```
$ stack build
```

==========

== JavaScript LSP with Flow ==

Flow: https://github.com/facebook/flow
Installation: https://flow.org/en/docs/install/

Uses Facebook's **flow** static type checker.

1) Install flow

```
# If using globally
$ npm install --global flow-bin

# If using locally, make sure in PATH
$ npm install flow-bin
$ export PATH='./node_modules/.bin:$PATH'
```

2) Set LSP configuration for flow
lspconfig: https://github.com/neovim/nvim-lspconfig/blob/master/doc/server_configurations.md#flow

```
lua <<EOF
local nvim_lsp = require('lspconfig')

vim.lsp.enable("flow")
vim.lsp.config("flow", {
```

```
  -- default
  cmd = { "npx", "--no-install", "flow", "lsp" }
  filetypes = { "javascript", "javascriptreact",
"javascript.jsx" }

  root_dir = function(bufnr, on_dir)
    local fname = vim.api.nvim_buf_get_name(bufnr)
    on_dir(nvim_lsp.util.root_pattern(".flowconfig")(fname))
  end,
  -- [DEPRECATED: < nvim 0.11]
  root_dir = nvim_lsp.util.root_pattern(".flowconfig")

  enabled = true
})

nvim_lsp.flow.settings = {
  -- default
  cmd = { "npx", "--no-install", "flow", "lsp" }
  filetypes = { "javascript", "javascriptreact",
"javascript.jsx" }
  root_dir = nvim_lsp.util.root_pattern(".flowconfig")
  enabled = true
}
EOF

set omnifunc=v:lua.vim.lsp.omnifunc
```

3) Configure project to use flow
flowconfig: https://flow.org/en/docs/config/
Run 'flow init' or make sure .flowconfig exists

```
$ flow init
$ ls | grep .flowconfig
# Make sure all=true
# .flowconfig
[ignore]


[include]

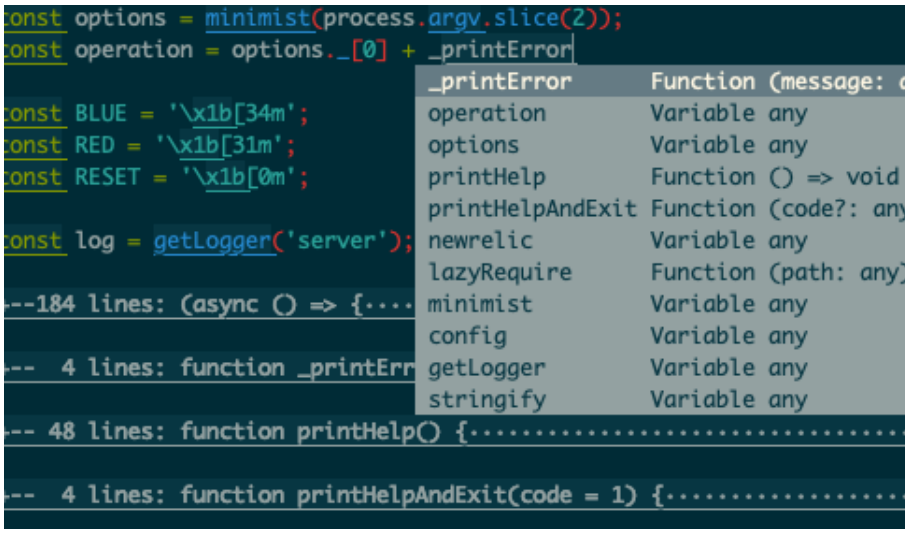

[libs]


[lints]


[options]
all=true


[strict]
```

4) Test
Open a JS file and try completion
Try completion with <C-O>



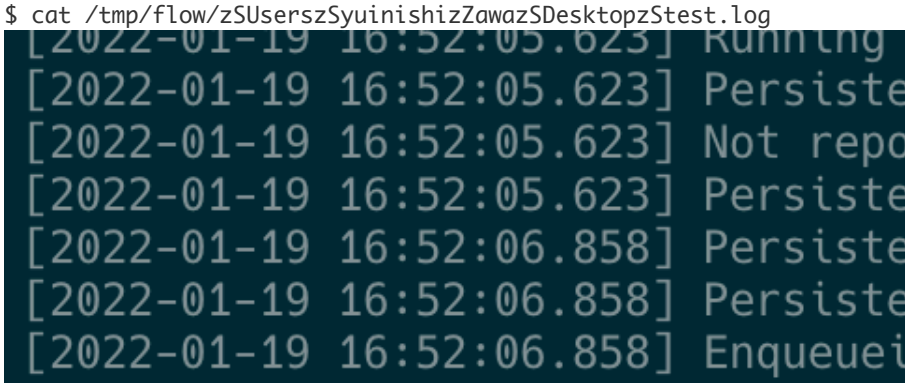
TROUBLE SHOOTING: check logs
Logs are under /tmp/flow/*.logs

```
# Log for ~/Desktop/test/index.js
$ cat /tmp/flow/zSUserszSyuinishizZawazSDesktopzStest.log
```

TROUBLE SHOOTING: flow not working. Log says "File is missing @flow pragma"
Either need @pragma or all=true option

e.g.

```
# Check error
$ cat /tmp/flow/zSUserszSyuinishizZawazSDesktopzStest.log
```



Set pragma
Working (notice the errors).



==========

== JavaScript LSP with TypeScript server ==

[DEPRECATED: use **ts_ls** instead] Javascript LSP with TypeScript server
Ref: https://stackoverflow.com/questions/64868071/i-cannot-activate-nvims-built-in-lsps-for-javascript-and-html

Uses **tsserver**

1) Install TypeScript

```
# If using globally
$ npm install --global typescript typescript-language-server

# If using locally, make sure in PATH
$ npm install flow-bin
$ export PATH='./node_modules/.bin:$PATH'

$ tsserver            # tsserver should exist
```

2) Set LSP configuration
lspconfig: https://github.com/neovim/nvim-lspconfig/blob/master/doc/server_configurations.md#tsserver

```
lua <<EOF
   local nvim_lsp = require('lspconfig')
   nvim_lsp.tsserver.settings = {
    -- Default settings
    cmd = { "typescript-language-server", "--stdio"  }
    filetypes = { "javascript", "javascriptreact",
"javascript.jsx", "typescript", "typescriptreact",
"typescript.tsx" }
    init_options = {
      hostInfo = "neovim"
    }
    root_dir = nvim_lsp.util.root_pattern("package.json",
"tsconfig.json", "jsconfig.json", ".git")
```

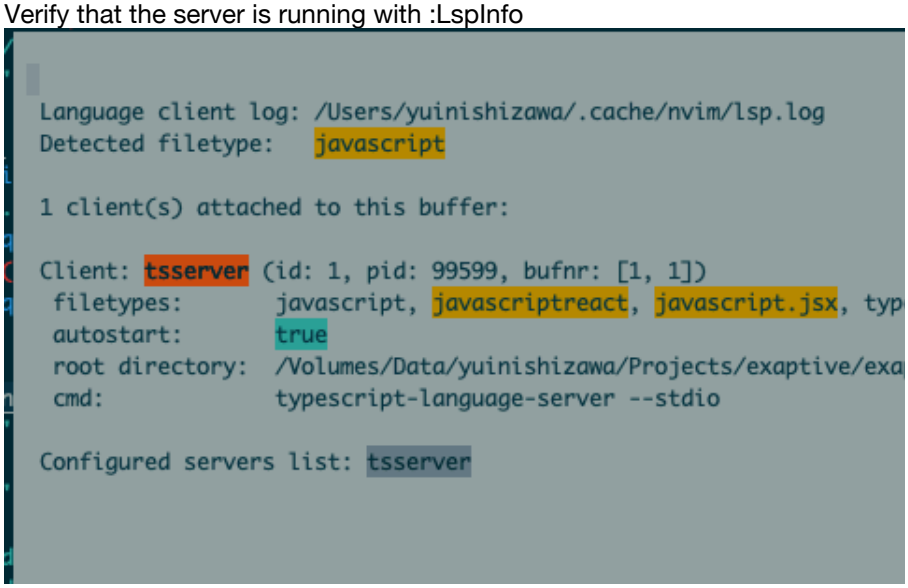
```
  }
EOF
```

3) Test
tsconfig: https://www.typescriptlang.org/tsconfig

Optionally create a tsconfig

```
# in ./.tsconfig
{
  "compilerOptions": {
    "module": "commonjs",
    "allowJs": true
  }
}
```

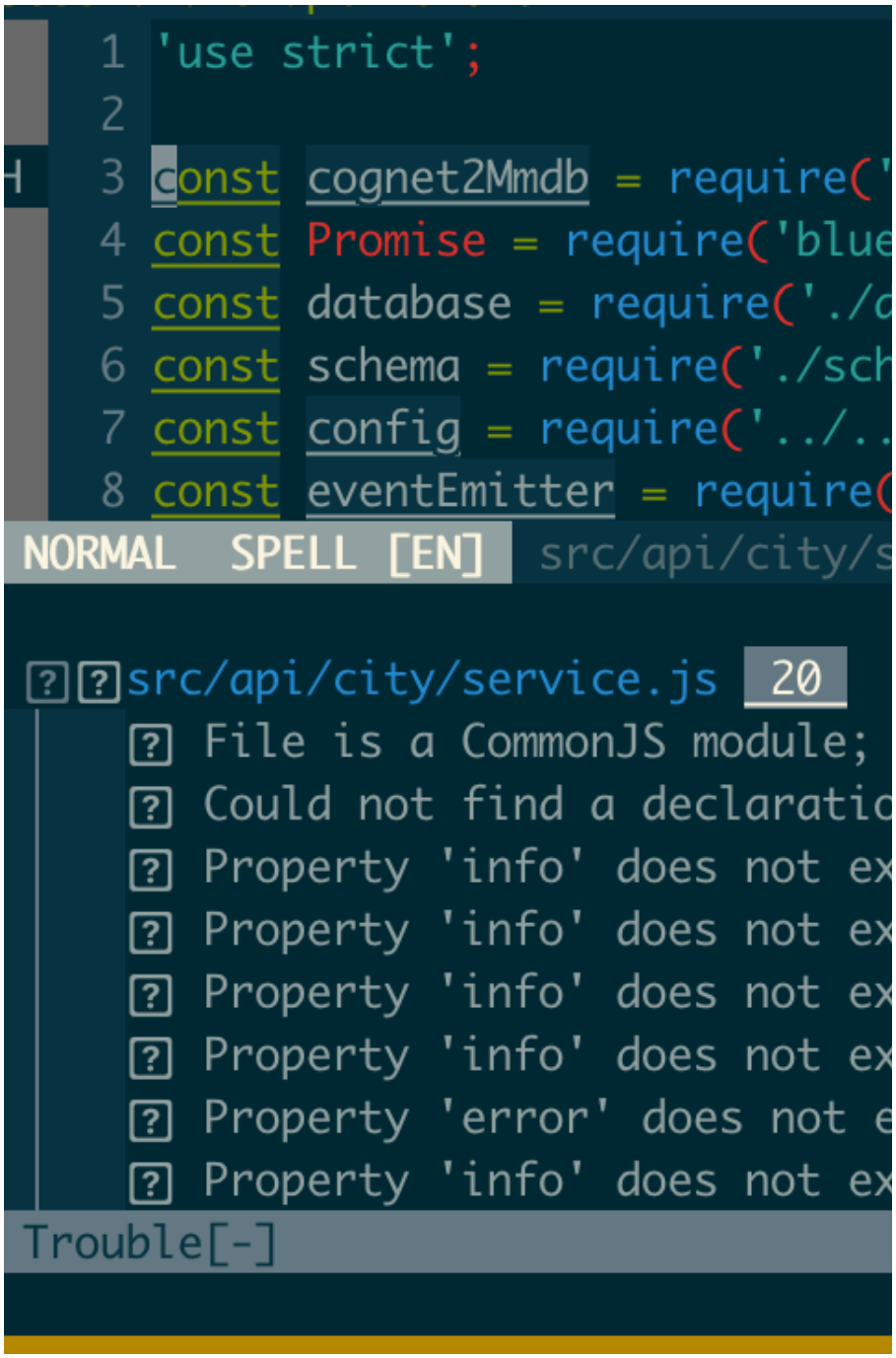
Verify that the server is running with :LspInfo



Try completion wit <C-O>



NOTE: lots of noise
tsserver notifies many TypeScript specific rules
Not yet sure how to handle this.
Either lspconfig or tsconfig?



TROUBLE SHOOTING: See configs used
Check the **tsserver logs**

Step 1) Update the **cmd**

```
lua <<EOF
  require'lspconfig'.tsserver.setup{
    cmd = { "typescript-language-server", "--stdio", "--log-
level", "4", "--tsserver-log-file", "/tmp/tsserver.txt" },
    ...
  }
EOF
```

Step 2) Open a file and check log.

```
# Should see something like
$ less /tmp/
Info 9    [15:26:57.022] Config: /Volumes/Data/yuinishizawa/
```

```
Projects/exaptive/exaptive_stack/repos/cognitive-city/test/
tsconfig.json : {
 "rootNames": [
  "/Volumes/Data/yuinishizawa/Projects/exaptive/exaptive_stack/
repos/cognitive-city/test/e2e/support/commands.ts",
  "/Volumes/Data/yuinishizawa/Projects/exaptive/exaptive_stack/
repos/cognitive-city/test/e2e/plugins/index.ts"
 ],
 "options": {
  "baseUrl": "/Volumes/Data/yuinishizawa/Projects/exaptive/
exaptive_stack/repos/cognitive-city/test"
  "configFilePath": "/Volumes/Data/yuinishizawa/Projects/exaptive/
exaptive_stack/repos/cognitive-city/test/tsconfig.json",
  ...
 }
}
```